Sistemas Inteligentes

Profa. Dra. Nathalia da Cruz Alves

O PERCEPTRON

A Combinação das Entradas - O “Net”

O somatório das entradas do neurônio multiplicadas pelos respectivos pesos sinápticos gera um valor intermediário *u*:

$$u = \sum x_i \cdot w_i + \theta$$

onde:

- x_i representa as entradas do neurônio
- w_i representa os pesos sinápticos
- θ representa o limiar de ativação
  - Em formulações modernas, costuma-se usar o bias *b*, onde *b* = - θ

Para duas entradas: $u = x_1 w_1 + x_2 w_2 + \theta$

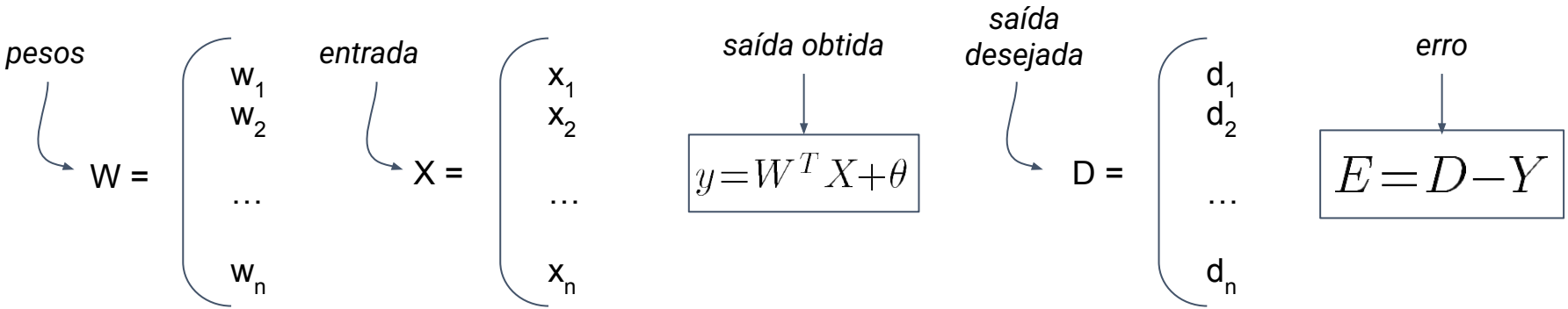
Após o cálculo de *u*, aplica-se uma função de ativação para produzir a saída do neurônio, representada por *y*:

$$y = \begin{cases} 1, \text{ se } u \geq \theta \\ 0, \text{ se } u < \theta \end{cases}$$

OBS: Em alguns materiais clássicos, *u* é chamado de *net (*soma ponderada ou valor líquido*)*

O PERCEPTRON (Aprendizado)

Formulação vetorial



- Suponha que temos um conjunto de **vetores de entrada** $\{x_1, x_2, ..., x_n\}$, cada um com seu **valor de saída correto** (desejado) $\{d_1, d_2, ..., d_n\}$
- O método de aprendizado busca encontrar os pesos de forma a **minimizar a diferença (erro) entre a saída desejada (*d*) e a saída obtida (*y*)** com o vetor de entrada

O PERCEPTRON (Aprendizado)

$$y = W^T X$$

$$f(u) = \begin{cases} 1, \text{ se } u > 0 \\ 0, \text{ se } u \leq 0 \end{cases}$$

$$E = D - Y$$

Na prática, w_0 representará θ e x_0 representará o valor do *bias*

O valor obtido (y) será utilizado como entrada na função de ativação

O erro será calculado a partir da diferença entre valor desejado e valor obtido

$$E = D - Y$$

saída desejada

valor *y* obtido pela rede

A	B	D
0	0	0
0	1	1
1	0	1
1	1	1

$$e = d - y$$

$$e(t) = d(t) - y(t)$$

a cada iteração *t*

O objetivo é **minimizar o erro**

O PERCEPTRON (Aprendizado)

Taxa de aprendizagem (*learning rate*)

- A taxa de aprendizagem representada por η (eta), é um valor entre 0 e 1 que controla o **quanto os pesos serão ajustados a cada iteração do treinamento**

Intuição:

- Ela define o “tamanho do passo” que o modelo dá na direção da correção do erro

Demonstração Perceptron com diferentes taxas de aprendizagem (0.001 e 1)

O PERCEPTRON (Aprendizado)

Taxa de aprendizagem (*learning rate*)

Taxa de aprendizagem muito pequena:

- aprendizado lento
- muitas iterações necessárias

Taxa de aprendizagem muito grande:

- pode oscilar
- pode não convergir

Uma taxa de aprendizagem adequada permite um aprendizado estável e eficiente

O PERCEPTRON (Aprendizado)

Regra de atualização do Perceptron

Para cada peso: $$w_i(t+1) = w_i(t) + \eta \cdot (d - y) \cdot x_i$$

onde:

η : taxa de aprendizagem
d : saída desejada
y : saída obtida
x_i : entrada
$(d - y)$: erro

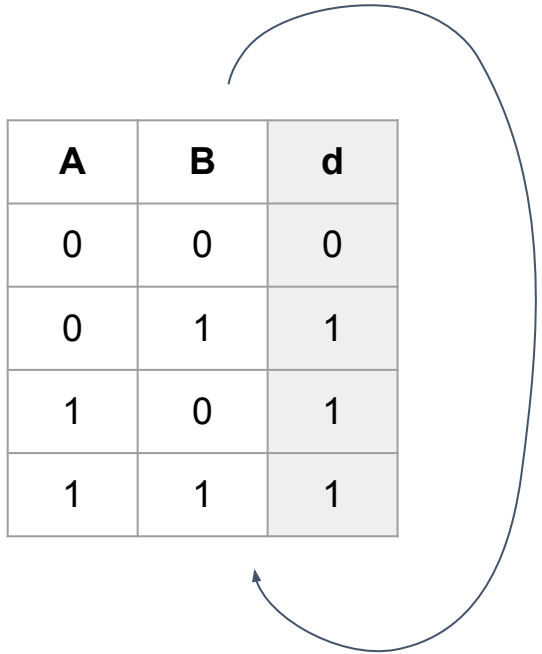
Para o *bias*: $$b(t+1) = b(t) + \eta \cdot (d - y)$$

O PERCEPTRON (Aprendizado)

Época (*epoch*)

Uma época corresponde a **uma passagem completa por todo o conjunto de dados de treinamento**

Ou seja, durante uma época, o modelo processa todas as amostras do conjunto de treinamento pelo menos uma vez

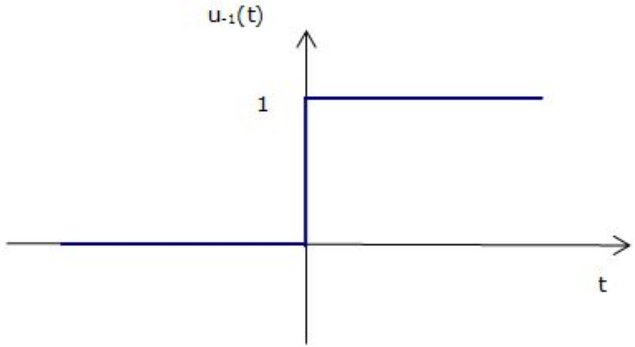


A	B	d
0	0	0
0	1	1
1	0	1
1	1	1

Função de Ativação ou de Saída

No Perceptron clássico, a função utilizada é a **função degrau**

$$f(u) = \begin{cases} 1, \text{ se } u > 0 \\ 0, \text{ se } u \leq 0 \end{cases}$$



- Simula uma classificação binária (ativada/não ativada)
- É simples computacionalmente

Em redes neurais modernas, funções contínuas e deriváveis são frequentemente utilizadas para permitir treinamento por gradiente

Algoritmo de Aprendizado do Perceptron

1. Início (t=0)
   - 1.1 – Definir valor de η entre 0 e 1.
   - 1.2 – Iniciar $\mathbf{w}(0)$ com valores nulos ou aleatórios.
2. Funcionamento
   - 2.1 – Selecionar vetor de entrada $\mathbf{x}(t)$.
   - 2.2 – Calcular ativação $u(t)$.
   - 2.3 – Calcular saída $y(t)$.
3. Treinamento
   - 3.1 – Calcular erro: $e(t) = d(t) - y(t)$
   - 3.2 – Ajustar pesos via regra de aprendizagem.
   - 3.3 – Verificar critério de parada.
     - 3.3.1 – Se atendido, finalizar treinamento.
     - 3.3.2 – Caso contrário, fazer t=t+1 e ir para Passo 2.

Algoritmo de Aprendizado do Perceptron

Exemplo Passo-a-Passo: Aprendendo a Porta Lógica OR.

Porta OR

x_1	x_2	d
0	0	0
0	1	1
1	0	1
1	1	1

t =0: Iniciar com zeros os pesos e o limiar.

$$w_1(0)=w_2(0)=\theta(0)=0$$

Logo:

$$\mathbf{w}(0)=\begin{bmatrix}\theta(0)\\ w_1(0)\\ w_2(0)\end{bmatrix}=\begin{bmatrix}0\\ 0\\ 0\end{bmatrix} \qquad \mathbf{x}(t)=\begin{bmatrix}-1\\ x_1(t)\\ x_2(t)\end{bmatrix}$$

Passo de aprendizagem escolhido: $\eta = 0{,}5$;

Algoritmo de Aprendizado do Perceptron

$$y = W^T X + \theta$$

$$f(u) = \begin{cases} 1, \text{ se } u > 0 \\ 0, \text{ se } u \leq 0 \end{cases}$$

$$E = D - Y$$

$$w_i(t+1) = w_i(t) + \eta \cdot (d - y) \cdot x_i$$

Calculando para a **primeira amostra - Época 1**:

t=1: Calcular saída para $\mathbf{w}(1) = [0 \;\; 0 \;\; 0]$ e $\mathbf{x}(1) = [-1 \;\; 0 \;\; 0]$.

$$u(1) = (0)(-1) + (0)(0) + (0)(0) = 0 \Rightarrow y(1) = 0, e(1) = 0.$$

$$\mathbf{w}(2) = \mathbf{w}(1) + \eta e(1)\mathbf{x}(1) = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} + (0{,}5)(0) \begin{bmatrix} -1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} = \mathbf{w}(1)$$

Observe que erro = 0 portanto, **não** há mudanças no vetor de pesos

Algoritmo de Aprendizado do Perceptron

$$y = W^T X + \theta$$

$$f(u) = \begin{cases} 1, \text{ se } u > 0 \\ 0, \text{ se } u \leq 0 \end{cases}$$

$$E = D - Y$$

$$w_i(t+1) = w_i(t) + \eta \cdot (d - y) \cdot x_i$$

Calculando para a **segunda amostra - Época 1**:

t=2: Calcular saída para $\mathbf{w}(2) = [0 \;\; 0 \;\; 0]$ e $\mathbf{x}(2) = [-1 \;\; 0 \;\; 1]$.

$u(2) = (0)(-1) + (0)(0) + (0)(1) = 0 \Rightarrow y(2) = 0$ $\quad e(2)=1$

$$\mathbf{w}(3) = \mathbf{w}(2) + \eta\, e(2)\, \mathbf{x}(2) = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} + (0{,}5)(1) \begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} -0{,}5 \\ 0 \\ 0{,}5 \end{bmatrix}$$

$$f(u) = \begin{cases} 1, \text{ se } u > 0 \\ 0, \text{ se } u \leq 0 \end{cases}$$

Observe que erro = 1 portanto, o vetor de pesos será atualizado

Algoritmo de Aprendizado do Perceptron

$$y = W^T X + \theta$$

$$f(u) = \begin{cases} 1, \text{ se } u > 0 \\ 0, \text{ se } u \leq 0 \end{cases}$$

$$E = D - Y$$

$$w_i(t+1) = w_i(t) + \eta \cdot (d - y) \cdot x_i$$

Calculando para a **terceira amostra - Época 1**:

$$f(u) = \begin{cases} 1, \text{ se } u > 0 \\ 0, \text{ se } u \leq 0 \end{cases}$$

t=3: Calcular saída para $\mathbf{w}(3) = [-0{,}5 \;\; 0 \;\; 0{,}5]$ e $\mathbf{x}(3) = [-1 \;\; 1 \;\; 0]$.

$u(3) = (-0{,}5)(-1) + (0)(1) + (0{,}5)(0) = 0{,}5 \Rightarrow y(3) = 1$, $e(1) = 0$

$\mathbf{w}(4) = \mathbf{w}(3)$

Observe que erro = 0 portanto, **não** há mudanças no vetor de pesos

Algoritmo de Aprendizado do Perceptron

$$y = W^T X + \theta$$

$$f(u) = \begin{cases} 1, \text{ se } u > 0 \\ 0, \text{ se } u \leq 0 \end{cases}$$

$$E = D - Y$$

$$w_i(t+1) = w_i(t) + \eta \cdot (d - y) \cdot x_i$$

Calculando para a **quarta amostra- Época 1**:

t=4: Calcular saída para $\mathbf{w}(4)$ = [-0,5 0 0,5] e $\mathbf{x}(4)$ =[-1 1 1].

$u(4) = (-0{,}5)(-1) + (0)(1) + (0{,}5)(1) = 1 \Rightarrow y(4) = 1$ $e(4)=0$

$\mathbf{w}(5) = \mathbf{w}(4)$

$$f(u) = \begin{cases} 1, \text{ se } u > 0 \\ 0, \text{ se } u \leq 0 \end{cases}$$

Observe que erro = 0 portanto, **não** há mudanças no vetor de pesos

Algoritmo de Aprendizado do Perceptron

$$y = W^T X + \theta$$

$$f(u) = \begin{cases} 1, \text{ se } u > 0 \\ 0, \text{ se } u \leq 0 \end{cases}$$

$$E = D - Y$$

$$w_i(t+1) = w_i(t) + \eta \cdot (d - y) \cdot x_i$$

Calculando para a **primeira amostra - Época 2**:

t=5: Calcular saída para $\mathbf{w}(5) = [-0{,}5 \;\; 0 \;\; 0{,}5]$ e $\mathbf{x}(5) = [-1 \;\; 0 \;\; 0]$.

$$f(u) = \begin{cases} 1, \text{ se } u > 0 \\ 0, \text{ se } u \leq 0 \end{cases}$$

$u(5) = (-0{,}5)(-1) + (0)(0) + (0{,}5)(0) = 0{,}5 \Rightarrow y(5) = 1$, $e(5) = -1$

$$\mathbf{w}(6) = \mathbf{w}(5) + \eta\, e(5)\mathbf{x}(5) = \begin{bmatrix} -0{,}5 \\ 0 \\ 0{,}5 \end{bmatrix} + (0{,}5)(-1)\begin{bmatrix} -1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0{,}5 \end{bmatrix}$$

Observe que erro = -1 portanto, o vetor de pesos será atualizado

Algoritmo de Aprendizado do Perceptron

$$y = W^T X + \theta$$

$$f(u) = \begin{cases} 1, \text{ se } u > 0 \\ 0, \text{ se } u \leq 0 \end{cases}$$

$$E = D - Y$$

$$w_i(t+1) = w_i(t) + \eta \cdot (d - y) \cdot x_i$$

Calculando para a **segunda amostra - Época 2**:

$$f(u) = \begin{cases} 1, \text{ se } u > 0 \\ 0, \text{ se } u \leq 0 \end{cases}$$

t=6: Calcular saída para $\mathbf{w}(6) = [0 \;\; 0 \;\; 0{,}5]$ e $\mathbf{x}(6) = [-1 \;\; 0 \;\; 1]$.

$u(6) = (0)(-1) + (0)(0) + (0{,}5)(1) = 0{,}5 \Rightarrow y(6) = 1$ $\quad e(6) = 0$

$\mathbf{w}(7) = \mathbf{w}(6)$

Observe que erro = 0 portanto, **não** há mudanças no vetor de pesos

Algoritmo de Aprendizado do Perceptron

$$y = W^T X + \theta$$

$$f(u) = \begin{cases} 1, \text{ se } u > 0 \\ 0, \text{ se } u \leq 0 \end{cases}$$

$$E = D - Y$$

$$w_i(t+1) = w_i(t) + \eta \cdot (d - y) \cdot x_i$$

Calculando para a **terceira amostra - Época 2**:

$$f(u) = \begin{cases} 1, \text{ se } u > 0 \\ 0, \text{ se } u \leq 0 \end{cases}$$

t=7: Calcular saída para $\mathbf{w}(7) = [0 \;\; 0 \;\; 0{,}5]$ e $\mathbf{x}(7) = [-1 \;\; 1 \;\; 0]$.

$u(7) = (0)(-1) + (0)(1) + (0{,}5)(0) = 0 \Rightarrow y(7) = 0$, $e(7) = 1$.

$$\mathbf{w}(8) = \mathbf{w}(7) + \eta\, e(7)\, \mathbf{x}(7) = \begin{bmatrix} 0 \\ 0 \\ 0{,}5 \end{bmatrix} + (0{,}5)(1) \begin{bmatrix} -1 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} -0{,}5 \\ 0{,}5 \\ 0{,}5 \end{bmatrix}$$

Observe que erro = 1 portanto, o vetor de pesos será atualizado

Algoritmo de Aprendizado do Perceptron

$$y = W^T X + \theta$$

$$f(u) = \begin{cases} 1, \text{ se } u > 0 \\ 0, \text{ se } u \leq 0 \end{cases}$$

$$E = D - Y$$

$$w_i(t+1) = w_i(t) + \eta \cdot (d - y) \cdot x_i$$

Calculando para a **quarta amostra - Época 2**:

$$f(u) = \begin{cases} 1, \text{ se } u > 0 \\ 0, \text{ se } u \leq 0 \end{cases}$$

t=8: Calcular saída para $\mathbf{w}(8) = [-0{,}5 \;\; 0{,}5 \;\; 0{,}5]$ e $\mathbf{x}(8) = [-1 \;\; 1 \;\; 1]$.

$u(8) = (-0{,}5)(-1)+(0{,}5)(1)+(0{,}5)(1) = 0{,}5 \Rightarrow y(8) = 1$ $e(8)=0$.

$\mathbf{w}(9) = \mathbf{w}(8)$

Observe que erro = 0 portanto, **não** há mudanças no vetor de pesos

Algoritmo de Aprendizado do Perceptron

t=9: Calcular saída para $\mathbf{w}(9) = [-0{,}5\ 0{,}5\ \ 0{,}5]$ e $\mathbf{x}(9) = [-1\ \ 0\ \ 0]$.

$u(9) = (-0{,}5)(-1) + (0{,}5)(0) + (0{,}5)(0) = 0{,}5 \Rightarrow y(9) = 1$, $e(9) = -1$.

$$\mathbf{w}(10) = \mathbf{w}(9) + \eta\, e(9)\, \mathbf{x}(9) = \begin{bmatrix} -0{,}5 \\ 0{,}5 \\ 0{,}5 \end{bmatrix} + (0{,}5)(-1) \begin{bmatrix} -1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0{,}5 \\ 0{,}5 \end{bmatrix}$$

t=10: Calcular saída para $\mathbf{w}(10) = [0\ \ 0{,}5\ \ 0{,}5]$ e $\mathbf{x}(10) = [-1\ \ 0\ \ 1]$.

$u(7) = (0)(-1) + (0{,}5)(0) + (0{,}5)(1) = 0{,}5 \Rightarrow y(10) = 1$, $e(10) = 0$.

$\mathbf{w}(11) = \mathbf{w}(10)$

t=11: Calcular saída para $\mathbf{w}(11) = [0\ \ 0{,}5\ \ 0{,}5]$ e $\mathbf{x}(11) = [-1\ \ 1\ \ 0]$.

$u(11) = (0)(-1) + (0{,}5)(1) + (0{,}5)(0) = 0{,}5 \Rightarrow y(11) = 1$, $e(11) = 0$.

$\mathbf{w}(12) = \mathbf{w}(11)$

Algoritmo de Aprendizado do Perceptron

t=12: Calcular saída para $\mathbf{w}(12) = [0 \;\; 0{,}5 \;\; 0{,}5]$ e $\mathbf{x}(12) = [-1 \;\; 1 \;\; 1]$.

$u(12)=(0)(-1)+(0{,}5)(1)+(0{,}5)(1)=1 \Rightarrow y(12)=1$, $e(12)=0$.

$\mathbf{w}(13)=\mathbf{w}(12)$

t=13: Calcular saída para $\mathbf{w}(13) = [0 \;\; 0{,}5 \;\; 0{,}5]$ e $\mathbf{x}(13) = [-1 \;\; 0 \;\; 0]$.

$u(13) = (0)(-1) + (0{,}5)(0) + (0{,}5)(0) = 0 \Rightarrow y(13) = 0$, $e(13)=0$.

$\mathbf{w}(14)=\mathbf{w}(13)$

FIM do treinamento!

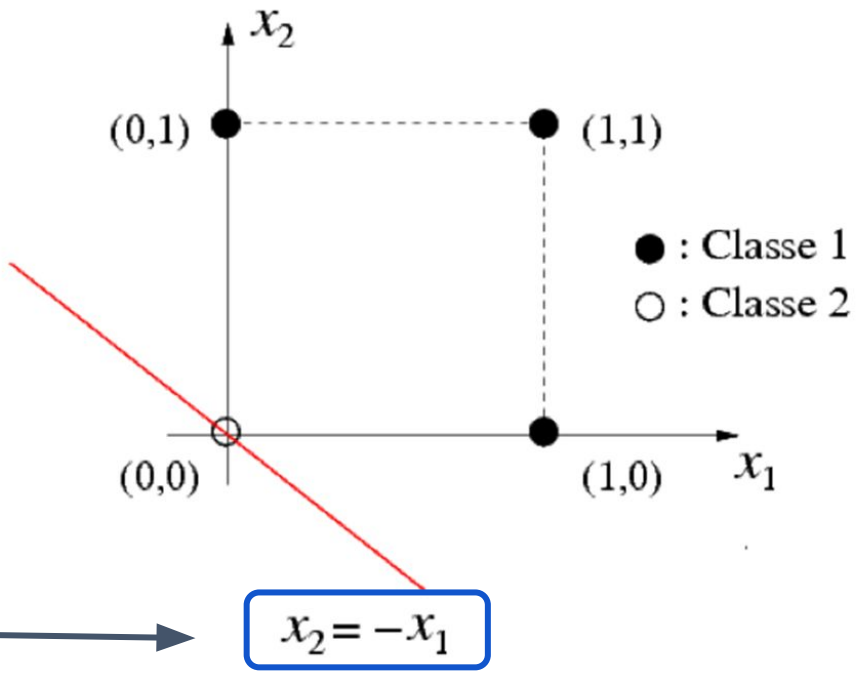
Algoritmo de Aprendizado do Perceptron

Solução Encontrada:

$$\mathbf{w} = \begin{bmatrix} \theta \\ w_1 \\ w_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0{,}5 \\ 0{,}5 \end{bmatrix}$$

$\theta + w_1 * x_1 + w_2 * x_2 = 0$

$0 + 0{,}5 * x_1 + 0{,}5 * x_2 = 0$



Exercício Perceptron

Um neurônio artificial do tipo Perceptron deve ser treinado para aprender o comportamento da porta lógica *AND*, que produz saída 1 somente quando as duas entradas são iguais a 1.

Considere que o neurônio possui três pesos:
bias θ, w_1 e w_2

A saída líquida do neurônio é calculada por:
$\theta + w_1 * x_1 + w_2 * x_2 = 0$

Considere a função de ativação degrau.

x_1	x_2	d
0	0	0
0	1	0
1	0	0
1	1	1

Considere os seguintes parâmetros iniciais: $\theta = 0$, $w_1 = 0$, $w_2 = 0$, $\eta = 0,1$

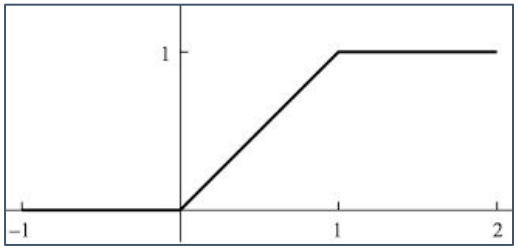
Para o bias, considere $x_0 = 1$

Apresente a execução do algoritmo de treinamento do Perceptron para encontrar os pesos finais.

Função de Ativação ou de Saída

Função rampa saturada

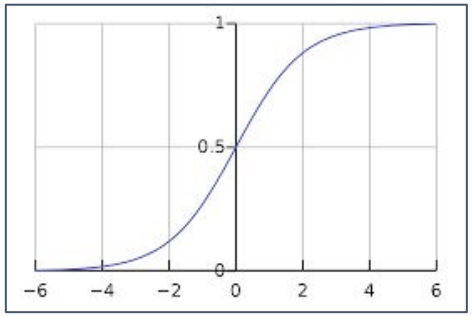
$$f(x)=\begin{cases}0, & x\leq 0\\ x, & 0\leq x<1\\ 1, & x\geq 1\end{cases}$$



- Saída entre 0 e 1
- Para valores negativos, não ativa
- Entre 0 e 1, cresce linearmente
- Satura em 1

Função Sigmoidal ou Logística

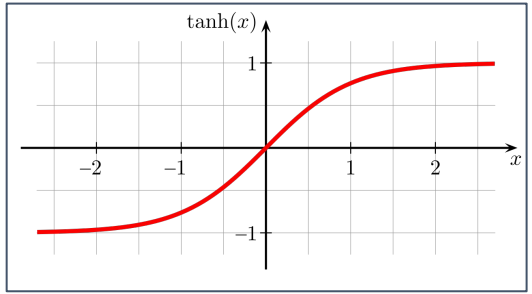
$$f(x)=\frac{1}{1+e^{-x}}$$



- Saída entre 0 e 1
- Muito usada em classificação binária
- Interpretação probabilística

Função Tangente Hiperbólica

$$\tanh(x)=\frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$$



- Saída entre -1 e 1
- Centragem em zero
- Frequentemente usada em camadas ocultas

O PERCEPTRON (O Renascimento)

- O Perceptron simples é capaz de resolver **apenas problemas linearmente separáveis**

- Apesar do descrédito enfrentado pela área de redes neurais, entre 1969 e 1982 os estudos neste campo continuaram, ainda que englobados em outras linhas de pesquisa

- Os primeiros resultados da retomada do desenvolvimento sobre redes neurais foram publicados em 1986 e 1987, por meio dos trabalhos do grupo PDP (*Parallel and Distributed Processing*), onde se consolidou a técnica de treinamento por *backpropagation*

Topologias de Redes Neurais

Como os neurônios se conectam?

- Um único neurônio ou elemento processador é **insuficiente** para a maioria dos problemas práticos, e, redes com grande número de neurônios interconectados são frequentemente utilizadas

- A maneira como os neurônios são interconectados determina as **capacidades computacionais da rede** e se constitui em uma das primeiras e mais importantes decisões no desenvolvimento de uma rede neural

Topologias de Redes Neurais

Redes diretas – *Feedforward* – MLP (*MultiLayer Perceptrons*)

- Não existem ciclos
- Conexões para a “frente”

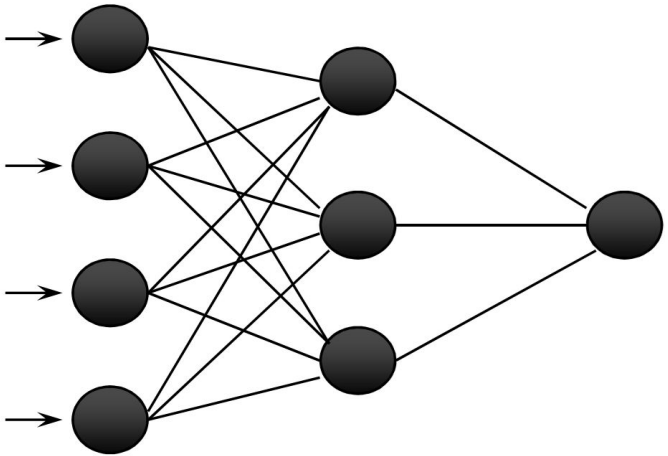
Redes recorrentes ou cíclicas

- Existem ciclos
- Conexões para trás e/ou para os lados

Topologias de Redes Neurais

Redes diretas – *Feedforward* – MLP (*MultiLayer Perceptrons*)

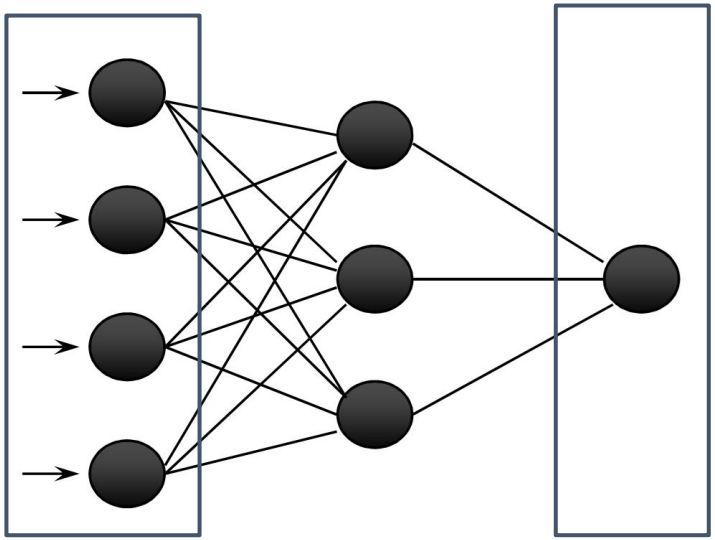
- É comum representar estas redes em camadas e, neste caso, são chamadas redes de camadas



Topologias de Redes Neurais

Redes diretas – *Feedforward* – MLP (*MultiLayer Perceptrons*)

Neurônios que recebem sinais de estímulo do ambiente são chamados de **camada de entrada** ou primeira camada



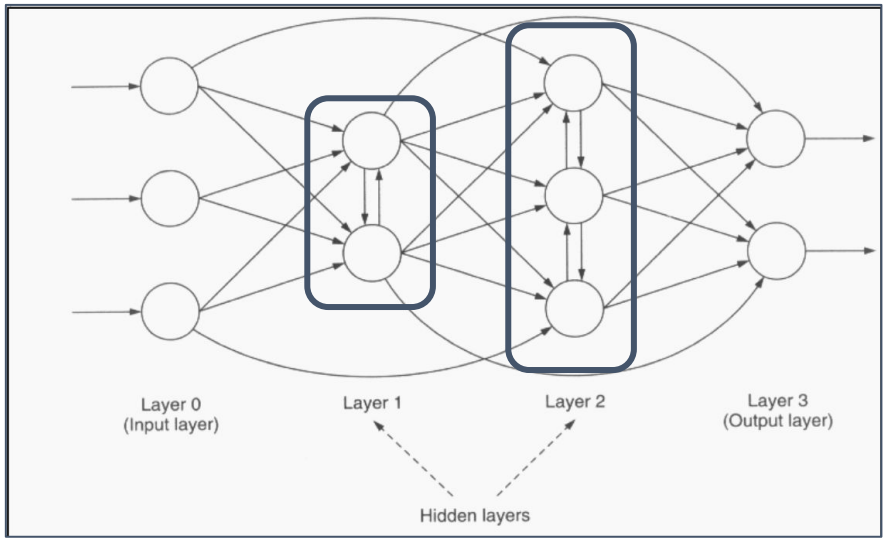
Neurônios que têm sua saída como saída da rede pertencem à **camada de saída** ou última camada

Neurônios que recebem sinais apenas de outros neurônios e enviam suas saídas para outros neurônios, pertencem às **camadas intermediárias** ou escondidas (*hidden*)

Topologias de Redes Neurais

Redes recorrentes ou cíclicas

Redes recorrentes são aquelas nas quais um neurônio da camada *i* pode possuir conexões com um ou mais neurônios de uma camada *j*, em que j ≤ i



Aprendizado de Redes Neurais

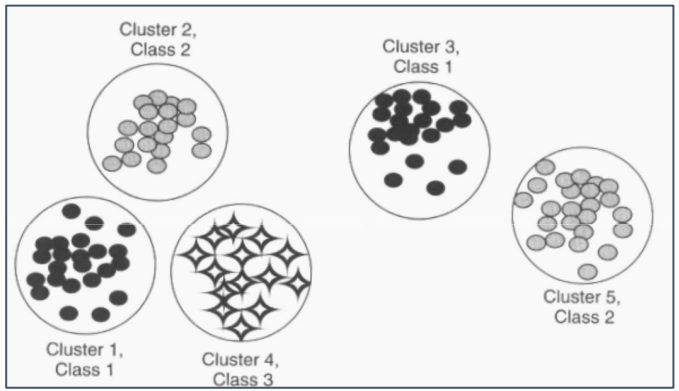
- Um neurônio é considerado um elemento **adaptativo**. Seus pesos sinápticos são modificáveis dependendo do algoritmo de aprendizado.

- Por exemplo, alguns, dependendo do sinal de entrada que recebem, têm seus valores de saída associados a uma resposta diante de um aprendizado supervisionado por uma espécie de "professor”

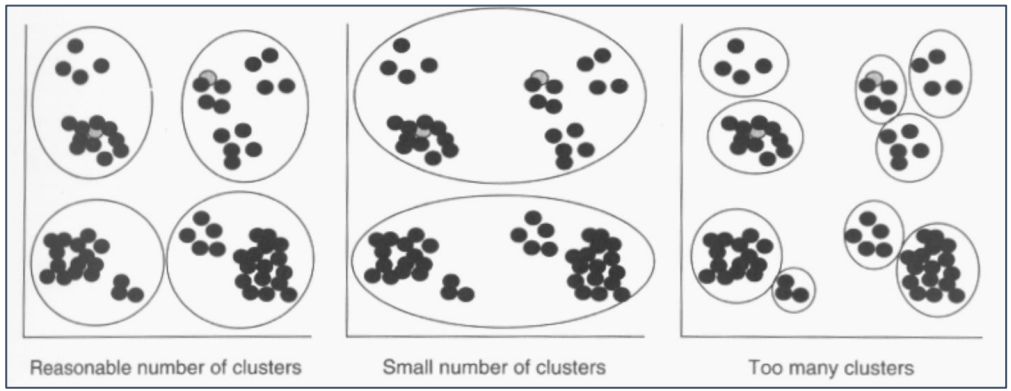
- Em alguns casos o sinal do "professor" não está disponível e não há informação de erro que possa ser utilizada para correção dos pesos sinápticos, assim o neurônio modificará seus pesos com base somente no sinal de entrada e/ou saída, sendo o caso do **aprendizado não-supervisionado**.

Aplicações de Redes Neurais

Classificação

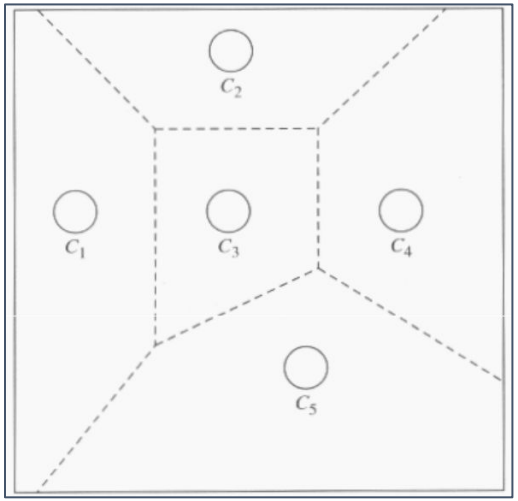


Agrupamento (*clustering*)

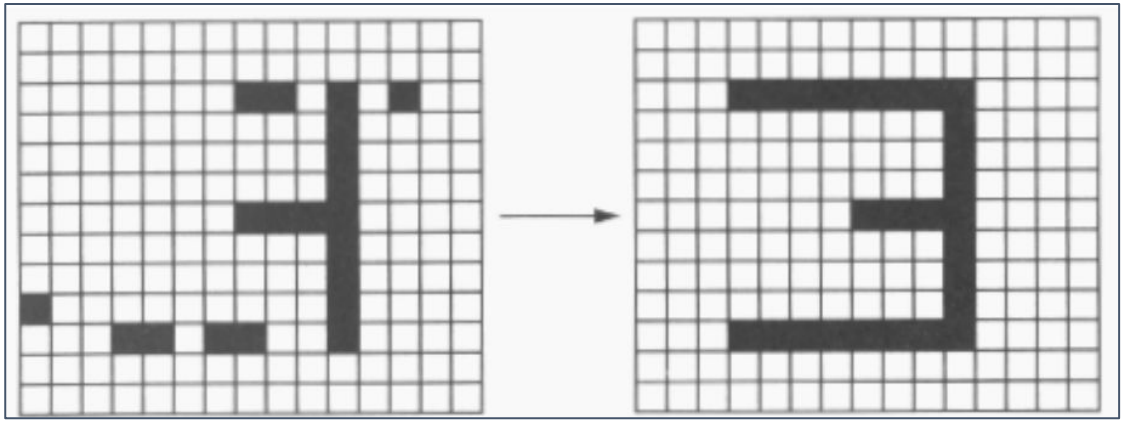


Aplicações de Redes Neurais

Quantização vetorial

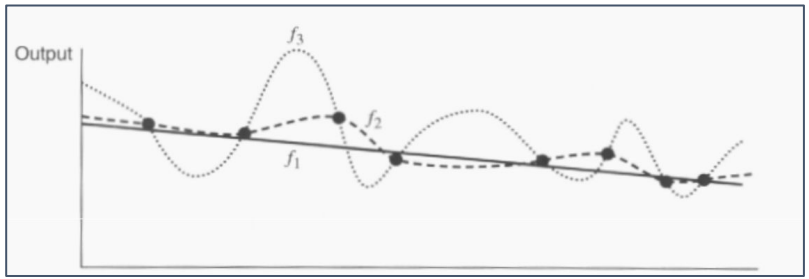


Associação de padrões

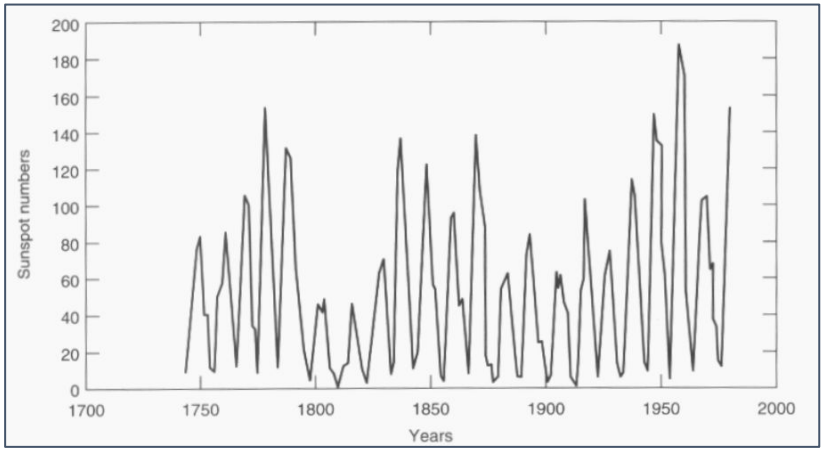


Aplicações de Redes Neurais

Aproximação de funções

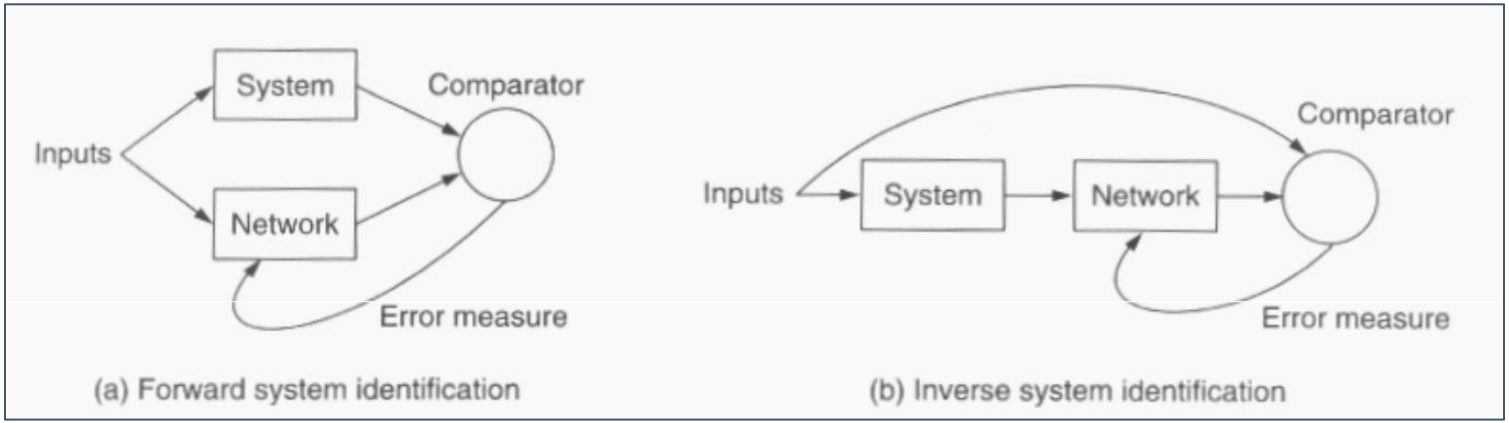


Previsões de séries temporais



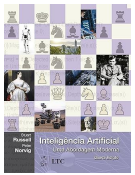
Aplicações de Redes Neurais

Aplicações em controle



Entre outros…

Leia mais sobre este tema...



Russell, S., Norvig, Peter. Inteligência Artificial - Capítulo 18. 2ª Ed. / Capítulo 19. 4ª Ed. Campus-Elsevier; Brasil, 2022.

creative commons

COMMONS DEED

Atribuição-Uso Não-Comercial-Compartilhamento pela Licença 2.5 Brasil

Você pode:

- copiar, distribuir, exibir e executar a obra

- criar obras derivadas

Sob as seguintes condições:

Atribuição — Você deve dar crédito ao autor original, da forma especificada pelo autor ou licenciante.

Uso Não-Comercial — Você não pode utilizar esta obra com finalidades comerciais.

Compartilhamento pela mesma Licença — Se você alterar, transformar, ou criar outra obra com base nesta, você somente poderá distribuir a obra resultante sob uma licença idêntica a esta.

Para ver uma cópia desta licença, visite http://creativecommons.org/licenses/by-nc-sa/2.5/br/ ou mande uma carta para Creative Commons, 171 Second Street, Suite 300, San Francisco, California, 94105, USA.